

NBA Player Stats Prediction System

Detailed explanation of how the project works, end to end

Generated on March 29, 2026



End-to-end flow



At a glance

- The main entry points are `update_data.py`, `train.py`, `simulate_season.py`, and `query_prob.py`.
- The data model is player-game centric, with team and opponent context attached to each row.
- The simulator uses injuries, lineups, defense, pace, correlations, and betting context.
- The query layer reuses cached projections and falls back to live simulation when needed.

1. What the project does

This project is an NBA player stat prediction system. It estimates points, rebounds, assists, steals, blocks, and turnovers for future games, then uses those projections to answer over/under questions and simulate entire slates. The repository is designed as a pipeline, not as one monolithic model. Each stage has a single job and a clear output that the next stage consumes.

- Raw data is collected from NBA Stats, injury feeds, betting lines, and lineup sources.
- Historical rows are converted into leakage-safe features that only use information available before the game being predicted.
- Per-stat machine learning models produce the core player projections.
- A simulation layer turns projections into game-level outcomes and player projection CSVs.
- A query layer lets you ask for over/under probabilities from the command line or interactively.

Key files

- `update_data.py` downloads and appends raw player and team logs.
- `train.py` drives the model training pipeline.
- `simulate_season.py` runs matchup and slate simulations.
- `query_prob.py` exposes the player probability CLI.
- `src/preprocessing/data_loader.py` merges player and team tables.
- `src/preprocessing/feature_engineer.py` builds the engineered feature set.
- `src/training/pipeline.py` trains CatBoost and Transformer models.
- `src/models/model_manager.py` loads models for inference and simulation.
- `src/simulation/game_simulator.py` performs Monte Carlo simulation.
- `src/query/interactive_cli.py` handles natural-language style queries.

Why the architecture matters

The system is split this way so the expensive jobs happen once and the cheap jobs happen repeatedly. Training runs offline on historical data. Simulation reuses saved models and cached context. Querying reuses exported projection files or can fall back to live simulation if a cached projection is missing. That separation keeps the workflow fast and makes the outputs easier to reason about.

2. Data ingestion and storage

The raw data pipeline starts in `update_data.py` and the scrapers under `src/data/`. The updater uses `nba_api` to fetch player game logs and team game logs, with rate limiting so NBA.com does not get hammered. It can work on the current season, a date range, the last ten seasons, or a full historical scrape. The script normalizes dates and computes fantasy points when that column is not already present.

- The code determines the current NBA season from the calendar month, so the default season changes automatically as the year changes.
- Existing CSVs in `data/` are read and merged with new rows so incremental updates do not require a full reload.
- Raw player logs land in `data/nba_players.csv` and game-level logs land in

data/nba_games.csv.

- The scrapers for injuries, lineups, betting lines, and defense are used later by the simulator and query flow.

DataLoader

src/preprocessing/data_loader.py is the bridge from raw CSVs to a merged player-game table. It parses minutes, sorts by player and date, and builds safe team-level rolling context. It also self-joins the team games table so the model can see opponent defense statistics that were available before the game.

- Minutes are normalized from either decimal or mm:ss formats.
- Team rolling features are computed with shift(1) so the current game never leaks into the past.
- The merge creates a single row per player game with the team context, opponent context, and game outcome fields attached.
- This is the row shape that the feature engineer and model training code expect.

The result is a consistent tabular dataset where each row represents a player in one game, with team and opponent context already aligned. That makes downstream feature engineering and model training much simpler.

3. Feature engineering

src/preprocessing/feature_engineer.py orchestrates the feature groups. It does not just add rolling averages. It combines a set of modular feature groups that capture recent form, efficiency, momentum, context, fatigue, matchup history, opponent strength, pace, role, target encoding, and league ranking. The file is built to keep the feature generation leakage-safe and diagnosable.

- Rolling windows use multiple horizons such as 3, 5, 10, 20, and 50 games.
- Momentum and fatigue features capture hot streaks, cold streaks, and rest patterns.
- Matchup features compare the player against the specific opponent.
- Opponent strength and pace features help the model understand game environment.
- Target encoding and league-ranking features add smoothed historical context without hard-coding raw identity effects.

Feature safety

The feature engineer sorts by PLAYER_ID and GAME_DATE, drops invalid dates, and tracks diagnostics about missing required and optional inputs. If the missingness is too high, the pipeline fails rather than silently training on broken data. That behavior is important because feature drift is one of the easiest ways for a sports model to become unreliable.

- FeatureSelector and FeatureSchema freeze the exact feature columns used during training so inference sees the same schema.
- Categorical columns such as PLAYER_ID, TEAM_ID, and OPPONENT_ID are tracked explicitly.
- The pipeline can save the feature schema to disk and reload it later for inference parity.

The training and prediction paths both depend on the same feature schema. That keeps the model from seeing a different column order or a different set of inputs after it is saved.

4. Training the models

train.py is the CLI front end for src/training/pipeline.py. The training pipeline splits the engineered dataset chronologically into fit, validation, and test partitions. It then trains one CatBoost regressor per target stat and optionally trains a Transformer sequence model on player histories. The quick, standard, and full modes mainly change iteration counts and epoch counts.

- CatBoost is the primary tabular learner because it handles mixed feature types well and is strong on structured sports data.
- Target-specific CatBoost profiles tune depth, learning rate, and regularization per stat instead of forcing one global setting.
- Quantile companions are trained so the system can estimate uncertainty, not just a point prediction.
- A second CatBoost model can act as an MAE companion for some targets.
- The Transformer learns from player sequences and contributes a second view of the same prediction problem.

Training flow

```
python update_data.py
python train.py --mode standard --parallel
python train.py --mode quick --no-gpu
python train.py --mode full --model-size large
```

TrainingPipeline first prepares the temporal split, then trains CatBoost across targets, then trains the Transformer if enabled. After validation, it builds inverse-MAE blend weights so the better model for a target gets more influence. The pipeline saves the feature schema, blend weights, per-target CatBoost artifacts, and the Transformer checkpoint into models/.

- models/feature_schema.pkl and models/feature_cols.pkl store the training schema.
- models/blend_weights.pkl stores the per-target CatBoost and Transformer blend ratios.
- models/attention_transformer.pkl stores the sequence model.
- Per-target CatBoost artifacts are saved with names like pts_catboost.cbm and supporting metadata files.

Compatibility note

src/pipeline/training_pipeline.py is only a compatibility shim. The active implementation lives in src/training/pipeline.py. That matters because older imports still work, but the real training logic is in the newer stack.

5. Simulation and reporting

simulate_season.py is the top-level entry point for game and slate simulation. It checks that trained models exist, builds a ModelManager, then wires together GameSimulator, ScheduleScraper, SeasonSimulator, and ReportGenerator. The simulation code is the part of the project that turns static projections into game outcomes.

- GameSimulator loads injuries, lineups, defense profiles, betting context, minutes estimates, and error calibration.
- The simulator uses a stat correlation matrix and a Cholesky factor so player stat outcomes move together realistically.
- Home court advantage, fatigue, clutch adjustments, and overtime logic are all modeled in the simulation layer.
- Roster context is cached because the same team context gets reused across many Monte

Carlo runs.

SeasonSimulator and ReportGenerator

SeasonSimulator batches games by date or slate and can use a thread pool on CPU, although GPU runs are intentionally kept single-threaded to avoid CUDA contention. ReportGenerator then summarizes the simulations, computes means, modes, and confidence ranges, and exports two CSVs: a game-level result file and a player projection file.

- simulate_season.py --today simulates all games for the current date.
- simulate_season.py --date YYYY-MM-DD targets a specific slate.
- simulate_season.py --week covers the next seven days.
- simulate_season.py --season simulates the remaining season schedule.

The player projection CSV is especially important because the query system reads it back later. That means a query can use a cached, precomputed projection rather than recomputing the whole simulation every time.

6. Query and probability calculation

query_prob.py is the user-facing CLI for over/under questions. In one-shot mode it loads the latest projections, finds the player, and calculates the probability of going over or under a betting line. In interactive mode, src/query/interactive_cli.py can also interpret natural-language style prompts and run a live simulation if the player is not in the cached file.

- QueryParser recognizes over/under phrasing, projection requests, compare commands, player listing, and exit/help commands.
- ProjectionLoader finds the latest player_projections_*.csv file and maps each row into a PlayerProjection object.
- ProbabilityCalculator converts a mean and standard deviation into over and under probabilities with a normal distribution.
- If a play probability is below 100 percent, the query result is adjusted to reflect the chance that the player actually plays.

Example queries

```
python query_prob.py -p "LeBron James" -s pts -l 25.5
python query_prob.py -p "Jokic" -s reb -l 12.5 --opponent DEN
python query_prob.py --compare "Curry" "Luka Doncic" --stat pts
python query_prob.py --list-players
```

The interactive CLI goes a step further than the simple probability calculator. It adjusts the projection mean based on recent form, minutes, matchup history, opponent defense, home or away context, and a rough player archetype. Then it runs a Monte Carlo draw so the final answer is not just a static percentage, but a distribution-aware estimate.

7. Configuration, outputs, and run order

config/default.yaml is the central place to inspect or tune behavior. It controls training targets, rolling windows, test split dates, simulation parameters, cache lifetimes, API endpoints, and default league averages. If you want to change how the project behaves without editing code, this is the first file to inspect.

- training.targets defines the six modeled stats: PTS, REB, AST, STL, BLK, and TOV.

- features and features_ext define rolling windows, EWMA spans, matchup history, and target encoding behavior.
- simulation controls the Monte Carlo engine, correlation settings, home edge, and overtime rules.
- cache controls how long schedule, betting, defense, lineup, and injury data stay fresh.
- api contains the NBA, Basketball Reference, ESPN, Action Network, and RotoWire endpoints used by the scrapers.

Common outputs

- data/nba_players.csv and data/nba_games.csv are the main raw inputs.
- models/ contains the trained CatBoost models, Transformer checkpoint, feature schema, and blend weights.
- data/sim_results/sim_results_*.csv stores slate or game summaries.
- data/sim_results/player_projections_*.csv stores player-level projection rows used by query_prob.py.

Recommended run order

```
python update_data.py
python train.py --mode standard --parallel
python simulate_season.py --today
python query_prob.py -p "Player Name" -s pts -l 22.5
```

That sequence is the intended loop: refresh data, train models, simulate the slate, then query the exported projections. The repo is built so each stage can be rerun independently, but the outputs are most useful when they are executed in that order.

8. Mental model for the whole system

A useful way to think about the project is as a cascade of narrowing uncertainty. Raw logs become a clean merged dataset. The merged dataset becomes engineered features. The features become model predictions. The predictions become simulation draws. The simulation draws become game summaries and over/under probabilities. Every file and class in the repo exists to move information cleanly from one stage to the next.